UNIVERSIDAD DE ANTIOQUIA
DEPARTAMENTO DE ELECTRÓNICA Y TELECOMUNICACIONES
2598521 - INFORMATICA II

Implementación de Tetris mediante manipulación de bits en C++

Informe de Análisis y Diseño Final

Rigoberto Berrio Berrio
Estudiante de Ingeniería Electrónica
Universidad de Antioquia
rigoberto.berrio1@udea.edu.co

Felipe Herrera Lopera
Estudiante de Ingeniería Electrónica
Universidad de Antioquia
felipe.herreral@udea.edu.co

23 de marzo de 2026

Índice

1. Análisis del problema y consideraciones de la solución **3**
1.1. Contexto del desafío 3
1.2. Restricciones y decisiones de diseño 3
1.3. Estrategia general de solución 3

2. Esquema de tareas y arquitectura del sistema **4**
2.1. Estructura modular 4
2.2. Diagrama de dependencias 4
2.3. Secuencia de tareas del game loop 4

3. Algoritmos implementados **7**
3.1. Representación de piezas con `uint16_t` (`piezas.cpp`) 7
3.2. Tablero como arreglo dinámico de bytes (`tablero.cpp`) 7
3.3. Lógica de juego (`juego.cpp`) 7
3.4. Entrada segura (`entrada.cpp`) 7

4. Problemas de desarrollo afrontados **8**

5. Evolución de la solución y consideraciones de implementación **8**
5.1. Evolución del diseño 8
5.2. Consideraciones prácticas de implementación 8

6. Conclusiones **9**

1. Análisis del problema y consideraciones de la solución

1.1 Contexto del desafío

El desafío consiste en diseñar e implementar una versión simplificada del juego Tetris en C++, ejecutado completamente desde la consola dentro del entorno Qt Creator. A diferencia del Tetris tradicional, el flujo es por turnos: el sistema muestra el tablero, solicita una acción al usuario y procesa el resultado del movimiento de la pieza actual.

La restricción central del problema esta en que la lógica de representación y manipulación de datos esté sustentada en operaciones a nivel de bits (*bitwise*), gestión eficiente de memoria dinámica mediante punteros, y programación modular distribuida en múltiples archivos `.h` y `.cpp`.

1.2 Restricciones y decisiones de diseño

El análisis inicial del problema identificó las siguientes restricciones para la arquitectura de la solución:

- **Sin clases ni objetos:** toda la organización de datos se realiza mediante `struct`s y funciones atómicas.
- **Sin STL ni `string`:** las colecciones y textos se manejan con arreglos estáticos y punteros a `char`.
- **Operadores bitwise obligatorios:** las piezas, el tablero y las colisiones se resuelven con máscaras, desplazamientos y operaciones `&`, `|`, `^`, `«`, `»`.
- **Memoria dinámica:** el tablero se construye en tiempo de ejecución con `malloc` según las dimensiones elegidas por el usuario.
- **Modularidad multi-archivo:** cada componente lógico vive en su propio par `.h`/`.cpp`.

1.3 Estrategia general de solución

La solución se estructura en cinco módulos independientes..

Para las **piezas**, se eligió representar cada rotación como un valor `uint16_t` de 16 bits, aprovechando que cualquier tetrimino cabe exactamente en una caja de 4×4 celdas (16 bits). Para el **tablero**, cada fila es un arreglo de bytes `uint8_t` donde cada bit representa una celda ocupada o libre, lo que permite detectar filas completas comparando directamente contra `0xFF`.

2. Esquema de tareas y arquitectura del sistema

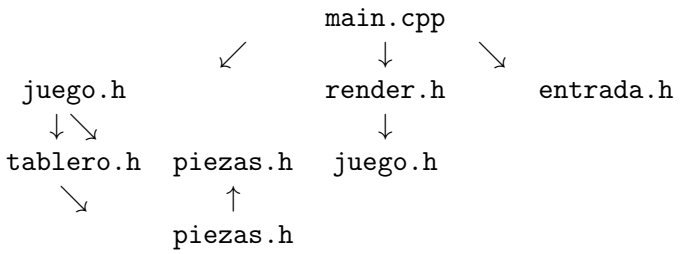
2.1 Estructura modular

El sistema se organiza en seis archivos fuente más el punto de entrada `main.cpp`. La Tabla 2.1 resume la responsabilidad de cada módulo y sus dependencias.

Módulo	Responsabilidad	Depende de
`piezas.h / .cpp`	Representación *bitwise* de las 7 piezas y sus rotaciones	—
`tablero.h / .cpp`	Tablero dinámico, colisiones, fijación de piezas, limpieza de filas	piezas.h
`juego.h / .cpp`	Estado global del juego, lógica de turnos y puntuación	piezas.h, tablero.h
`render.h / .cpp`	Visualización del tablero y la HUD en consola	juego.h
`entrada.h / .cpp`	Lectura segura del teclado y configuración del tablero	—
`main.cpp`	Inicialización y game loop principal	juego.h, render.h, entrada.h

Cuadro 1: Módulos del sistema y sus dependencias

2.2 Diagrama de dependencias



2.3 Secuencia de tareas del game loop

1. El programa solicita al usuario el ancho y alto del tablero con validación estricta: solo se aceptan enteros, el ancho debe ser múltiplo de 8.
2. Se inicializa el estado del juego: tablero vacío en memoria dinámica, pieza actual y pieza siguiente generadas aleatoriamente centradas en el tablero.
3. En cada turno: se limpia la pantalla, se dibuja el tablero con la pieza activa superpuesta, y se muestra la puntuación y siguiente pieza.
4. Se lee la acción del usuario. Solo las teclas `A`, `D`, `S`, `W`, `Q` son reconocidas; cualquier otra entrada es descartada sin efecto.
5. Se procesa la acción: movimiento lateral, bajada, rotación o salida. La pieza solo

baja si el usuario presiona `S`.

6. Si la pieza no puede bajar más, se fija en el tablero. Se detectan y eliminan filas completas, y se actualiza la puntuación.
7. Se genera la siguiente pieza. Si no puede colocarse en la parte superior, se activa el Game Over y el loop termina.
8. Se libera toda la memoria dinámica del tablero antes de salir del programa.

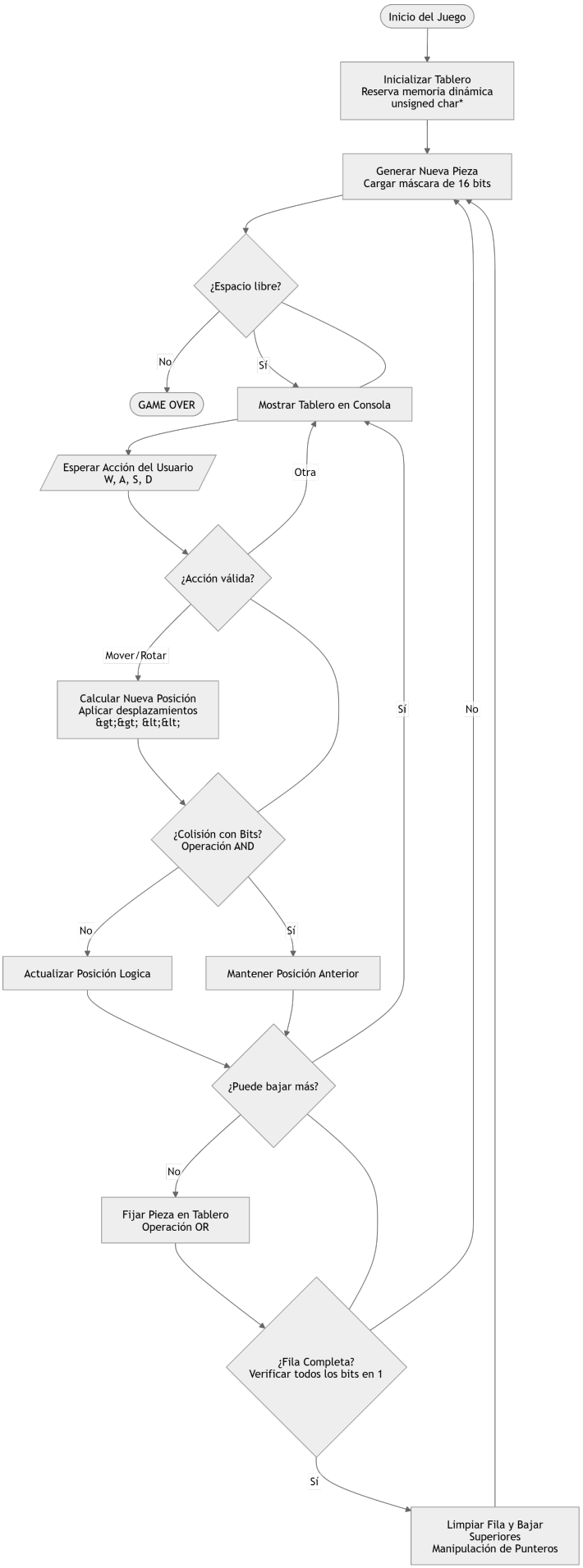


Figura 1: Diagrama del Sistema de Juego

3. Algoritmos implementados

3.1 Representación de piezas con `uint16_t` (`piezas.cpp`)

Cada pieza de Tetris se representa como un valor de 16 bits donde cada grupo de 4 bits corresponde a una fila de la caja 4×4. El bit 15 es la celda (fila 0, col 0) y el bit 0 es la celda (fila 3, col 3). Extraer si una celda está ocupada es una operación de un solo desplazamiento y una máscara AND.

La rotación se implementa mediante tablas de permutación de bits (`ROT_ORIGEN` / `ROT_DESTINO`), que mapean cada bit de su posición original a su posición rotada.

Las formas base de las 7 piezas se almacenan en una tabla estática dentro de `piezas.cpp`, invisible para los demás módulos.

3.2 Tablero como arreglo dinámico de bytes (`tablero.cpp`)

El tablero se representa como `uint8_t**`: un arreglo dinámico de punteros, donde cada puntero apunta a una fila de `bytesPerFila = ancho/8` bytes. Un bit en 1 indica celda ocupada. La inicialización reserva toda la memoria.

La detección de colisiones itera sobre los bits activos de la pieza y verifica, para cada uno, si la celda destino está dentro de los límites y libre.

La limpieza de filas aprovecha la representación en bytes: una fila completa tiene todos sus bytes en `0xFF`. En lugar de copiar datos, se reasignan punteros, lo que es O(1) por fila desplazada.

3.3 Lógica de juego (`juego.cpp`)

La función `procesarTurno` centraliza toda la lógica de un turno. Distingue cada acción posible y aplica gravedad solo cuando el usuario presiona `S`. Si la pieza no puede bajar, la fija, limpia filas, actualiza la puntuación y verifica si la nueva pieza puede generarse (Game Over).

El sistema de puntuación sigue la tabla clásica de Tetris:

Filas eliminadas	Puntos otorgados
1	100
2	300
3	500
4	800

Cuadro 2: Puntuación según filas eliminadas por turno

3.4 Entrada segura (`entrada.cpp`)

La lectura de enteros desde consola usa `getchar()` carácter por carácter, verificando que cada uno sea un dígito ASCII (código entre 48 y 57) antes de convertir. Si el

usuario ingresa letras u otros caracteres, la función devuelve `-1` y el `do-while` vuelve a pedir el valor sin dejar basura en el buffer.

La normalización a minúsculas de las teclas se hace con la operación bitwise `accion | 0x20`, que activa el bit 5 del código ASCII, convirtiendo cualquier mayúscula en su equivalente minúscula sin necesidad de condicionales.

4. Problemas de desarrollo afrontados

1. **Bajada involuntaria de la pieza con cualquier tecla.** El problema original en `entrada.cpp` forzaba cualquier tecla no reconocida a `ACCION_BAJAR`, haciendo que la pieza bajara con cualquier entrada. La corrección fue devolver `0` (acción nula) para teclas desconocidas.
2. **Loop infinito al ingresar letras en** `pedirAncho` / `pedirAlto`**.** `scanf("%d", &ancho)` no consume el carácter inválido del buffer cuando recibe una letra, provocando que el `do-while` lea el mismo carácter indefinidamente. La solución fue reemplazar `scanf` por `leerEnteroSeguro()`, que lee la línea completa con `getchar()` y valida cada carácter antes de convertir.
3. **Limpieza de filas con doble liberación de memoria.** Al eliminar una fila completa y reasignar punteros, existía el riesgo de perder el puntero original antes de liberarlo. Se corrigió liberando primero con `free(tablero->filas[f])` y luego corriendo los punteros hacia abajo, asegurando que no haya doble liberación.

5. Evolución de la solución y consideraciones de implementación

5.1 Evolución del diseño

5.2 Consideraciones prácticas de implementación

- **Tipos de datos:** se eligió `uint16_t` para las piezas porque 16 bits encajan exactamente con una caja 4×4, eliminando bits de relleno y simplificando las operaciones. Se eligió `uint8_t` para las filas del tablero porque el ancho siempre es múltiplo de 8, garantizando que no hay bits desperdiciados.
- **Memoria dinámica:** el tablero usa `uint8_t**` (puntero a arreglo de punteros) en lugar de un bloque contiguo 2D, lo que permite reasignar punteros al eliminar filas sin copiar datos, logrando desplazamiento O(1) por fila.
- **Escalabilidad:** agregar un nuevo módulo al proyecto solo requiere declarar las funciones en un nuevo `.h`, implementarlas en el `.cpp` correspondiente.

6. Conclusiones

La implementación demuestra que es posible desarrollar un motor de juego funcional en C++ sin utilizar clases, STL ni `string`, mediante el uso de operaciones bitwise, aritmética de punteros y una adecuada gestión de memoria dinámica.

La representación de las piezas con `uint16_t` resultó eficiente, ya que permite manejar rotaciones y simplifica la detección de colisiones a comparaciones de bits con el tablero.

Además, la arquitectura modular en múltiples archivos facilitó el trabajo, permitiendo el desarrollo independiente de los módulos y una integración más sencilla, con errores localizados sin afectar otros componentes.